

1 Introduction to Pipelining

Pipelining is an implementation technique in which multiple instructions are overlapped in execution, much like an assembly line.

- **Core Concept:** Just as a car assembly line works on multiple cars simultaneously (one getting an engine, another getting wheels), a pipelined processor works on different stages of multiple instructions at the same time.
- **Goal:** To increase instruction **throughput**.
- **Throughput:** The number of instructions completed per unit of time.
- **Latency:** The time required to complete a single instruction.

The Pipelining Paradox

Pipelining does **NOT** decrease the time it takes to execute a single instruction (latency). In fact, it often slightly increases it due to the overhead of pipeline control and registers. However, because we are finishing more instructions per second, the **total execution time for a program decreases**.

1.1 The Laundry Analogy

Lecture 4.5 uses a laundry analogy to explain pipelining. Imagine you have four loads of laundry (A, B, C, D) and four stages: **Wash, Dry, Fold, Store**. Each stage takes 30 minutes.

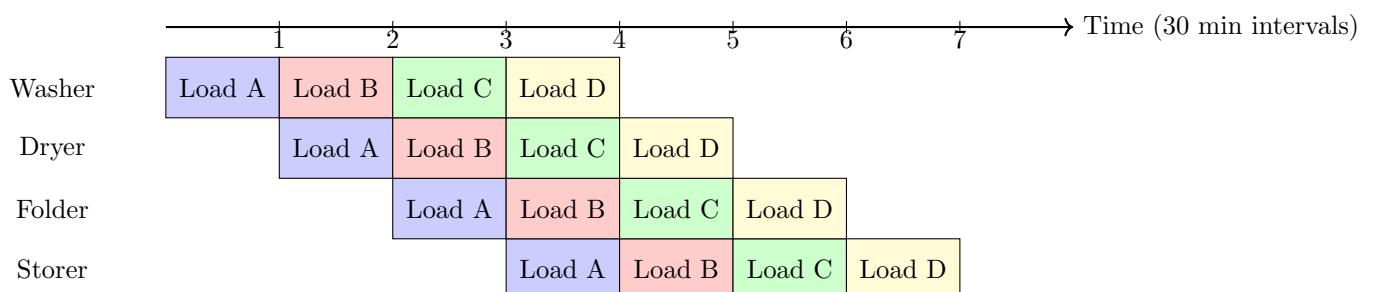
1.1.1 Sequential vs. Pipelined Execution

1. Sequential (Non-Pipelined): You wait for Load A to finish all 4 stages (2 hours) before starting Load B.

- Load A: 6:00 - 8:00
- Load B: 8:00 - 10:00 ...
- **Total Time for 4 loads:** 4×2 hours = 8 hours.

2. Pipelined:

- **6:00:** Load A into Washer.
- **6:30:** Move Load A to Dryer. Put Load B into Washer.
- **7:00:** Move A to Fold, B to Dryer. Put C into Washer.
- **Total Time for 4 loads:** 3.5 hours.



Note: At Time=3.5, all stages are busy!

Figure 1: Visualizing Pipelined Laundry (Ideal Case)

2 The classic Five-Stage RISC-V Pipeline

Most RISC-V implementations use a 5-stage pipeline.

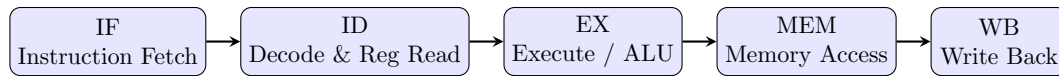


Figure 2: The 5 Stages of RISC-V Pipeline

1. **IF (Instruction Fetch):** Fetch the instruction from memory using the Program Counter (PC).
2. **ID (Instruction Decode):** Decode the instruction and read source registers from the Register File.
3. **EX (Execute):** Perform the operation (ALU) or calculate a memory address.
4. **MEM (Memory Access):** Read/Write data from/to Data Memory (only for Loads/Stores).
5. **WB (Write Back):** Write the result back into the Register File.

Performance Math: Speedup Formula

If all stages are perfectly balanced, the potential speedup is equal to the **number of pipeline stages**.

$$\text{Speedup} = \frac{\text{Pipeline Depth}}{1 + \text{Pipeline Stalls}} \quad (1)$$

In our ideal laundry example with 4 stages, the speedup approaches 4x as the number of loads increases.

- **Single-Cycle Clock:** Must be long enough for the *slowest instruction* (e.g., 800ps).
- **Pipelined Clock:** Must be long enough for the *slowest stage* (e.g., 200ps).

3 Design for Pipelining

Why is RISC-V so good for pipelining compared to x86?

- **Fixed Length (32-bit):** IF and ID stages are simple and predictable. We know where the next instruction is immediately ($PC + 4$).
- **Regular Formats:** Source registers (**rs1**, **rs2**) are always in the same bit positions. We can read registers *while* decoding.
- **Load-Store Architecture:** Calculation and Memory Access are separated. We don't have complex instructions like `ADD [Memory], Register` that would require a complex loop in the pipeline.

4 Pipeline Hazards

A **Hazard** is why we can't always just pipeline everything perfectly. It prevents the next instruction from executing in the next cycle.

4.1 1. Structural Hazards

Problem: Hardware resource conflict.

Analogy: You only have one robotic arm that does both putting clothes in the washer AND folding. It can't do both at once.

Computer Example: If instruction memory and data memory were the same single block, we couldn't Fetch (IF) an instruction while another instruction Accesses Memory (MEM).

Solution: Separate Instruction Memory (IMem) and Data Memory (DMem), or separate L1 caches.

4.2 2. Data Hazards

Problem: An instruction needs data that hasn't been written back yet.

Example Code:

```
add x19, x0, x1    // Produces a value for x19
sub x2, x19, x3    // Needs that value of x19 IMMEDIATELY
```

The `sub` instruction is in the ID stage while `add` is only in EX. The value won't be written to the register file until `add` reaches WB.

4.2.1 Solution A: Forwarding (Bypassing)

Don't wait for the data to be written to the register! Grab it directly from the ALU output buffer.

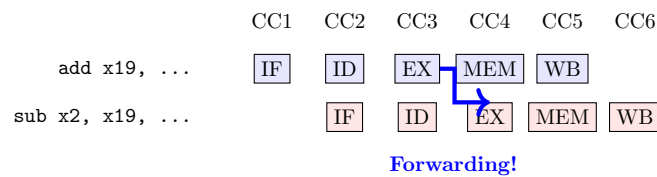


Figure 3: Visualizing Forwarding: Data goes from ADD's EX directly to SUB's EX.

4.2.2 Solution B: Stalling (Load-Use Hazard)

If the producing instruction is a **Load**, the data isn't ready until after MEM. Forwarding from EX isn't possible because the data isn't there yet! We **must** wait one cycle. This is a **Bubble**.

4.3 3. Control Hazards

Problem: Deciding which instruction to fetch next.

Context: Conditional branches (`beq`, `bne`). The processor fetches the next instruction *before* it knows if the branch is taken or not.

Solution:

- **Stall:** Wait until outcome is known (insert bubbles). Simple but slow.
- **Predict:** Guess the outcome (e.g., "Always Not Taken").
 - If right: No penalty.
 - If wrong: Flush pipeline (cancel instructions), restart.
- **Dynamic Prediction:** Hardware remembers recent behavior (Taken/Not Taken) to guess future behavior. Accuracy > 90%.
- **Delayed Branch (MIPS):** The architecture automatically executes the instruction *immediately following* the branch (in the "delay slot") regardless of the branch outcome. The compiler fills this slot with a safe instruction.

5 Flashcards

1. **Q: What is pipelining in the context of computer architecture?**
A: It is an implementation technique in which multiple instructions are overlapped in execution, much like an assembly line.
2. **Q: What is the primary performance goal of pipelining?**
A: Pipelining aims to improve instruction throughput, which is the number of instructions completed per unit of time.
3. **Q: What is the 'pipelining paradox' regarding instruction execution time?**
A: The time to complete a single instruction (latency) is not shorter, but the total time for many instructions is shorter due to increased throughput.
4. **Q: Under ideal conditions with a large number of instructions, the speed-up from pipelining is approximately equal to what?**
A: The number of pipe stages.
5. **Q: What are the five classic stages of a RISC-V instruction pipeline?**
A: 1. Fetch instruction, 2. Decode instruction/Read registers, 3. Execute, 4. Access memory, 5. Write back result.
6. **Q: In a pipelined processor, what determines the duration of the clock cycle?**
A: The duration of the slowest pipeline stage.
7. **Q: In a non-pipelined, single-cycle processor, what determines the duration of the clock cycle?**
A: The time required to execute the slowest possible instruction.
8. **Q: What is a pipeline hazard?**
A: A situation in pipelining when the next instruction cannot execute in the following clock cycle for some reason.
9. **Q: Events in a pipeline such as start-up and ----- affect performance when the number of tasks is not large compared to the number of stages.**
A: wind-down
10. **Q: What is a structural hazard in a pipeline?**
A: A situation where a planned instruction cannot execute because the hardware cannot support the combination of instructions set to execute in the same clock cycle.
11. **Q: What is a data hazard in a pipeline?**
A: A situation where an instruction cannot execute in the proper clock cycle because data it needs is not yet available from a previous instruction.
12. **Q: What is a control hazard in a pipeline?**
A: A situation where the proper instruction cannot execute because the instruction that was fetched is not the one that is needed, typically due to a branch.
13. **Q: What is another common name for a control hazard?**
A: A branch hazard.
14. **Q: How does having a fixed length for all RISC-V instructions simplify pipelining?**
A: It makes it much easier to fetch instructions in the first pipeline stage and decode them in the second stage.
15. **Q: How does RISC-V being a load-store architecture simplify pipelining?**
A: It allows the memory address calculation and memory access to occur in predictable, separate stages (EX and MEM).
16. **Q: A specific type of data hazard where data from a load instruction is needed by a subsequent instruction before it's available is called a -----.**
A: load-use data hazard
17. **Q: What is forwarding, also known as bypassing, used for?**
A: It is a method of resolving a data hazard by retrieving the missing data from internal buffers rather than waiting for it to be written to the register file.

18. **Q: Why is forwarding sometimes insufficient, such as in a load-use data hazard?**
A: The desired data is not available until after the memory access stage, which is too late for the execution stage of the immediately following instruction.
19. **Q: A stall initiated in order to resolve a hazard is often called a ____.**
A: bubble
20. **Q: Besides hardware stalls, what is a software technique a compiler can use to mitigate data hazards?**
A: Reordering code to separate a dependent instruction from the one that produces its data.
21. **Q: What is the simplest, albeit slow, hardware solution to a control hazard caused by a branch?**
A: To stall the pipeline after fetching a branch until the branch outcome and target address are known.
22. **Q: A method of resolving a branch hazard by assuming an outcome and proceeding rather than waiting is known as ____.**
A: branch prediction
23. **Q: What is a simple static branch prediction strategy mentioned in the text?**
A: To predict that conditional branches will always be untaken.
24. **Q: What is dynamic branch prediction?**
A: A technique where hardware makes predictions based on the past behavior of each individual conditional branch, and may change predictions during program execution.
25. **Q: What must the pipeline control do when a branch prediction is incorrect?**
A: It must ensure the wrongly fetched instructions have no effect and restart the pipeline from the correct branch address.
26. **Q: An architectural solution to control hazards, used by MIPS, that always executes the instruction immediately following a branch is called a ____.**
A: delayed branch
27. **Q: Pipelining improves instruction ____ rather than instruction ____.**
A: throughput, latency
28. **Q: What is pipeline latency?**
A: The number of stages in a pipeline, representing the time it takes for a single instruction to complete its execution.
29. **Q: In the RISC-V single-cycle example, the clock cycle was 800 ps because it had to be stretched to accommodate the slowest instruction, which was ____.**
A: ld (load doubleword)
30. **Q: In the graphical representation of forwarding in Figure 4.27, the path goes from the output of the EX stage of the add instruction to the input of the EX stage of which instruction?**
A: The sub instruction.
31. **Q: In the code reordering example, which instruction was moved up to eliminate the pipeline stalls?**
A: The third ld instruction (ld x4, 16(x31)).
32. **Q: Why is it impossible for a forwarding path to go from the MEM stage of one instruction to the EX stage of the *following* instruction without a stall?**
A: This would mean going backward in time, as the following instruction's EX stage occurs at the same time as the current instruction's MEM stage.
33. **Q: Assuming conditional branches are 17% of instructions, what is the CPI if every branch causes a one-cycle stall and all other instructions have a CPI of 1?**
A: The CPI would be 1.17.
34. **Q: In a pipeline diagram, what does shading on the right half of the register file symbol indicate?**
A: It indicates that the register file is being read in that stage.

35. **Q: In a pipeline diagram, what does shading on the left half of the register file symbol indicate?**
A: It indicates that the register file is being written in that stage.
36. **Q: The text states that modern implementations of the x86 architecture often translate complex x86 instructions into simpler, RISC-V-like operations before pipelining them. Why is this done?**
A: Because x86 instructions vary in length from 1 to 15 bytes, making direct pipelining considerably more challenging.
37. **Q: In what pipeline stage does a RISC-V instruction write its result?**
A: The last stage (Write-back or WB).
38. **Q: In the example from Figure 4.24, what was the single-cycle clock time and what was the pipelined clock time?**
A: The single-cycle time was 800 ps (for the slowest instruction, ld), and the pipelined time was 200 ps (for the slowest stage, memory access or ALU).
39. **Q: What is a classic example of a structural hazard in a processor pipeline?**
A: When one instruction is accessing data memory while another instruction is being fetched from memory, if there is only a single memory unit.
40. **Q: In the sequence add x19, x0, x1 followed by sub x2, x19, x3, how does forwarding resolve the data hazard?**
A: The result from the ALU in the EX stage of the add instruction is forwarded directly to the ALU input for the EX stage of the sub instruction.
41. **Q: A forwarding path is only valid if the destination stage is ----- in time than the source stage.**
A: later
42. **Q: In the example $a = b + e$; $c = b + f$;, how can the code be reordered to avoid pipeline stalls?**
A: By moving the load for f up, between the loads for b and e and the first add instruction, to separate the dependent instructions.
43. **Q: What happens in a 'predict-not-taken' scheme if the branch is actually taken?**
A: The pipeline stalls, and the incorrectly fetched instructions are flushed while the pipeline restarts from the correct branch address.
44. **Q: What is the key difference between static and dynamic branch prediction?**
A: Static prediction is rigid (e.g., always predict taken for backward branches), while dynamic prediction uses the past behavior of each specific branch to make its guess.
45. **Q: What is the typical accuracy of modern dynamic branch predictors?**
A: They can correctly predict conditional branches with more than 90% accuracy.
46. **Q: For the code sequence: ld x10, 0(x10) followed by add x11, x10, x10, can it execute without stalling?**
A: No, this is a load-use hazard that must stall for one cycle, even with forwarding.
47. **Q: For the code sequence: add x11, x10, x10 followed by addi x12, x11, 5, can it execute without stalling?**
A: Yes, this data hazard can be resolved using only forwarding.
48. **Q: For the code sequence: addi x11, x10, 1 followed by addi x12, x10, 2 and addi x13, x10, 3, can it execute without issues?**
A: Yes, it can execute without stalling or forwarding because there are no data dependencies between the instructions.
49. **Q: If one stage in a pipeline is slower than the others, what effect does it have?**
A: The slowest stage determines the clock cycle time for the entire pipeline.
50. **Q: Ideally, what is the speedup from pipelining equal to?**
A: The number of stages in the pipeline.

51. **Q: Why do x86 architectures translate their instructions before pipelining?**
A: Because x86 instructions vary in length from 1 to 15 bytes, which makes standard pipelining very difficult; they are translated into simpler, fixed-length micro-operations.
52. **Q: In the structural hazard example with a single memory, which two pipeline stages collide?**
A: The IF stage (fetching instruction $i + 3$) and the MEM stage (accessing data for instruction i).
53. **Q: In the code reordering example ($a = b + e; c = b + f$), how many cycles are saved by moving the load for 'f' up?**
A: 2 cycles (it eliminates two 1-cycle stalls).
54. **Q: What is the colloquial nickname for a pipeline stall mentioned in the text?**
A: A "bubble".
55. **Q: What is the specific calculated CPI penalty for "Stall on Branch" if 17% of instructions are branches?**
A: 1.17 (Calculated as $1 + 0.17 \times 1$ stall cycle).
56. **Q: Which specific architecture uses the "Delayed Branch" technique where the instruction after a branch is always executed?**
A: The MIPS architecture.
57. **Q: In a Delayed Branch system, where does the compiler place a "safe" instruction?**
A: Immediately after the branch instruction (in the branch delay slot).
58. **Q: What does the text say about the visibility of pipelining to the programmer?**
A: It is fundamentally invisible to the programmer (unlike multiprocessor programming).
59. **Q: Define 'Instruction Throughput'.**
A: The quantitative measure of the number of instructions completed per unit of time by a processor.
60. **Q: Define 'Pipeline Latency'.**
A: The time interval required for a single instruction to traverse all stages of the pipeline and complete execution.
61. **Q: What is a 'Pipeline Stall' (Bubble)?**
A: A temporary suspension of instruction fetching and execution, effectively inserting NOPs (No Operations), to resolve a hazard.
62. **Q: Clearly distinguish between 'Forwarding' and 'Bypassing'.**
A: They are synonymous terms. Both refer to the hardware technique of routing a data result from an internal pipeline buffer (e.g., ALU output) directly to a functional unit input, skipping the Register File write-read cycle.
63. **Q: What technically constitutes a 'Load-Use Data Hazard'?**
A: A specific data dependency where an instruction requires an operand that is being loaded from memory by the immediately preceding instruction. The data is not available in the pipeline until the MEM stage completes.
64. **Q: What is 'Branch Prediction Miss Penalty'?**
A: The number of clock cycles wasted when a branch outcome is mispredicted, requiring the pipeline to be flushed and refilled from the correct target address.
65. **Q: Define 'Static Branch Prediction'.**
A: A prediction strategy that uses fixed heuristics (e.g., "backward branches are usually taken") and does not change based on execution history.
66. **Q: Define 'Dynamic Branch Prediction'.**
A: A prediction strategy that uses hardware structures (like Branch History Tables) to track the past outcomes of branches and predict future behavior based on that history.
67. **Q: technically, why does a 'Control Hazard' occur?**
A: Because the Program Counter (PC) must be updated in the IF stage, but the condition for a branch instruction is often not resolved until the ID or EX stage.

68. Q: What is the 'Branch Delay Slot' in MIPS architecture?

A: The instruction position immediately following a branch instruction. This instruction is executed regardless of the branch outcome to utilize the cycle that would otherwise be a stall.

69. Q: What is the function of the 'IF/ID Register'?

A: A pipeline register that holds the instruction fetched from memory and the PC value, preserving them for the Decode stage while the Fetch stage retrieves the next instruction.

70. Q: What is 'PC-Relative Addressing' in the context of branches?

A: A method where the branch target address is calculated by adding an immediate offset to the current Program Counter (PC) value.

71. Q: In RISC-V, what specific hazard is NOT resolved by forwarding alone?

A: The Load-Use Data Hazard. It requires a one-cycle stall even with forwarding.